```
"""
manifest_parser.py
==================
GRUS Audit Pipeline — Stage 1: Manifest Parsing
Specification: GRUS-STD-001-v1.0 Section 3 Stage 1

Validates that a submission's GRUS_MANIFEST.yaml file exists, parses
correctly, and contains all required fields. Returns the parsed manifest
or raises a structured failure report.

Published by: Green Recursive Utility Service LLC
"""

import os
import yaml
from typing import Dict, Any
from dataclasses import dataclass


@dataclass
class ManifestValidationResult:
    """Result of manifest parsing stage."""
    passed: bool
    manifest: Dict[str, Any]
    failure_reason: str = ""
    failing_field: str = ""


REQUIRED_FIELDS = [
    "submission_title",
    "contributor_name",
    "contributor_type",  # individual | classroom | team | institution
    "doi",
    "doi_timestamp",
    "constants_used",
    "datasets_cited",
    "predictions",
    "license",
]

OPTIONAL_FIELDS = [
    "submitter_pseudonym",
    "affiliation",
    "previous_audit_ids",
    "publication_request",  # public | private (default: public on pass)
]
```

```
def parse_manifest(repo_path: str) -> ManifestValidationResult:
    """
    Parse and validate GRUS_MANIFEST.yaml in the submission repository root.

    Args:
        repo_path: absolute path to submission repository root

    Returns:
        ManifestValidationResult
    """
    manifest_path = os.path.join(repo_path, "GRUS_MANIFEST.yaml")

    if not os.path.exists(manifest_path):
        return ManifestValidationResult(
            passed=False,
            manifest={},
            failure_reason="GRUS_MANIFEST.yaml not found in repository root",
            failing_field="manifest_file",
        )

    try:
        with open(manifest_path, "r", encoding="utf-8") as f:
            manifest = yaml.safe_load(f)
    except yaml.YAMLError as e:
        return ManifestValidationResult(
            passed=False,
            manifest={},
            failure_reason=f"YAML parse error: {e}",
            failing_field="manifest_yaml_syntax",
        )
    except Exception as e:
        return ManifestValidationResult(
            passed=False,
            manifest={},
            failure_reason=f"Manifest read error: {e}",
            failing_field="manifest_file",
        )

    if not isinstance(manifest, dict):
        return ManifestValidationResult(
            passed=False,
            manifest={},
            failure_reason="Manifest must be a YAML mapping (dict)",
            failing_field="manifest_root",
        )

    # Validate required fields
    for field in REQUIRED_FIELDS:
        if field not in manifest:
            return ManifestValidationResult(
```

```
            passed=False,
            manifest=manifest,
            failure_reason=f"Required field missing: {field}",
            failing_field=field,
        )
    if manifest[field] in (None, "", []):
        return ManifestValidationResult(
            passed=False,
            manifest=manifest,
            failure_reason=f"Required field empty: {field}",
            failing_field=field,
        )

# Validate constants_used structure
constants = manifest.get("constants_used", [])
if not isinstance(constants, list):
    return ManifestValidationResult(
        passed=False,
        manifest=manifest,
        failure_reason="constants_used must be a list",
        failing_field="constants_used",
    )
for i, c in enumerate(constants):
    if not isinstance(c, dict):
        return ManifestValidationResult(
            passed=False,
            manifest=manifest,
            failure_reason=f"constants_used[{i}] must be a dict",
            failing_field=f"constants_used[{i}]",
        )
    for required_key in ("symbol", "from", "classification"):
        if required_key not in c:
            return ManifestValidationResult(
                passed=False,
                manifest=manifest,
                failure_reason=f"constants_used[{i}] missing key: {required_key}",
                failing_field=f"constants_used[{i}].{required_key}",
            )

# Validate datasets_cited structure
datasets = manifest.get("datasets_cited", [])
if not isinstance(datasets, list):
    return ManifestValidationResult(
        passed=False,
        manifest=manifest,
        failure_reason="datasets_cited must be a list",
        failing_field="datasets_cited",
    )
for i, d in enumerate(datasets):
    if not isinstance(d, dict) or "url" not in d:
```

```
            return ManifestValidationResult(
                passed=False,
                manifest=manifest,
                failure_reason=f"datasets_cited[{i}] must have 'url' field",
                failing_field=f"datasets_cited[{i}].url",
            )

    # Validate predictions structure
    predictions = manifest.get("predictions", [])
    if not isinstance(predictions, list):
        return ManifestValidationResult(
            passed=False,
            manifest=manifest,
            failure_reason="predictions must be a list",
            failing_field="predictions",
        )
    for i, p in enumerate(predictions):
        if not isinstance(p, dict):
            return ManifestValidationResult(
                passed=False,
                manifest=manifest,
                failure_reason=f"predictions[{i}] must be a dict",
                failing_field=f"predictions[{i}]",
            )
        # Forward-facing predictions must have falsification_condition (Clause 5)
        if p.get("status") == "pending":
            if "falsification_condition" not in p or not p["falsification_condition"]:
                return ManifestValidationResult(
                    passed=False,
                    manifest=manifest,
                    failure_reason=f"predictions[{i}] is pending but lacks
falsification_condition (Clause 5)",
                    failing_field=f"predictions[{i}].falsification_condition",
                )

    return ManifestValidationResult(
        passed=True,
        manifest=manifest,
        failure_reason="",
        failing_field="",
    )


def example_manifest() -> str:
    """Return an example GRUS_MANIFEST.yaml for documentation."""
    return """\
# GRUS_MANIFEST.yaml — submission manifest
# Required by GRUS-STD-001-v1.0

submission_title: "Rust formation rate from substrate viscous coupling"
```

```
contributor_name: "Mrs. Chen's Tenth-Grade Chemistry Class, Lincoln High School"
contributor_type: "classroom"  # individual | classroom | team | institution
affiliation: "Lincoln High School, Anytown TX"
doi: "10.5281/zenodo.NNNNNN"
doi_timestamp: "2026-05-15"
license: "MIT"

constants_used:
  - {symbol: "B", from: "svcf_constants.py", classification: "EXACT"}
  - {symbol: "ETA", from: "svcf_constants.py", classification: "MEASURED"}

datasets_cited:
  - url: "https://corrosion.nist.gov/data/example"
    description: "NIST corrosion database, iron samples"
    access_statute: "OSTP 2022 Public Access Mandate"

predictions:
  - id: "D49-residual"
    status: "confirmed"
    predicted_value: "rust rate at 80% humidity"
    observed_value: "from NIST dataset"
    residual: "<5%"
  - id: "P17"
    status: "pending"
    description: "Catalyst-family universality for analog corrosion systems"
    falsification_condition: "Transmission ratio > B = 96.97% in any system at >3σ"

publication_request: "public"
"""



if __name__ == "__main__":
    import sys
    if len(sys.argv) < 2:
        print("Usage: python manifest_parser.py <repo_path>")
        print("\nExample manifest:")
        print(example_manifest())
        sys.exit(0)

    result = parse_manifest(sys.argv[1])
    if result.passed:
        print("[PASS] Manifest validated.")
        print(f"  Title: {result.manifest.get('submission_title')}")
        print(f"  Contributor: {result.manifest.get('contributor_name')}")
    else:
        print(f"[FAIL] {result.failure_reason}")
        print(f"  Failing field: {result.failing_field}")
        sys.exit(1)
```